

AI Assistant Coding - 9.5

Name : E.ANJANI

HT. No : 2303A52419

Batch: 32

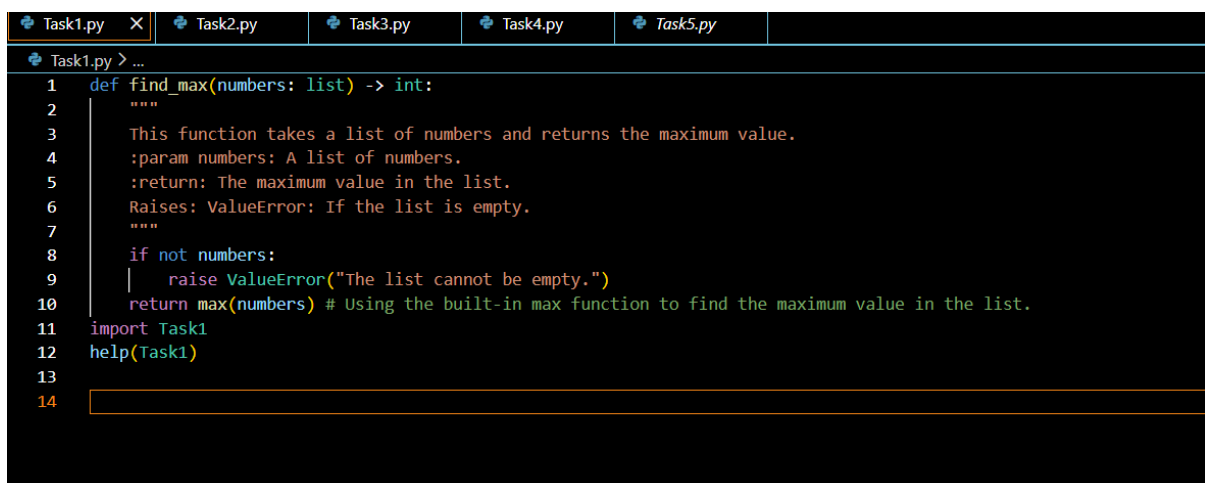
Problem 1: Consider the following Python function:

```
def find_max(numbers):  
    return max(numbers)
```

Task:

- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Code:



```
Task1.py x Task2.py Task3.py Task4.py Task5.py  
Task1.py > ...  
1 def find_max(numbers: list) -> int:  
2     """  
3     This function takes a list of numbers and returns the maximum value.  
4     :param numbers: A list of numbers.  
5     :return: The maximum value in the list.  
6     Raises: ValueError: If the list is empty.  
7     """  
8     if not numbers:  
9         raise ValueError("The list cannot be empty.")  
10    return max(numbers) # Using the built-in max function to find the maximum value in the list.  
11 import Task1  
12 help(Task1)  
13  
14
```

Advantages and Disadvantages:

Feature	Docstrings (PEP 257)	Inline Comments	Google-Style
Primary Audience	Users of the function	Maintainers of the code	Both Users and Maintainers
Advantages	Simple, fast to write, accessible via help().	Explains <i>why</i> a specific line exists; great for complex logic.	Highly structured, easy to read, supports auto-generation of docs.
Disadvantages	Lacks detail on types or specific error handling.	Can clutter code; doesn't show up in help() or IDE tooltips.	Verbose; can take up more space than the code itself.
Best Use Case	Small scripts or internal helper functions.	Explaining "hacks," workarounds, or complex algorithms.	Public APIs, libraries, and large-scale team projects.

Recommendation: Google Style is best one to use.

In Google style method we will mention what is input type and output type and who logic is gonna work.

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
    return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Code:

```
Task1.py Task2.py Task3.py Task4.py Task5.py
Task2.py > ...
1 def login(user: str, password: str, credentials: dict) -> bool:
2     """
3     Checks if the provided username and password match the credentials.
4     Args:
5         user (str): The username to check.
6         password (str): The password to check.
7         credentials (dict): A dictionary containing username-password pairs.
8     Returns:
9         bool: True if the credentials match, False otherwise.
10    Raises:
11        ValueError: If the password is not a string.
12    """
13    if user not in credentials:
14        return False
15    if not isinstance(password, str):
16        raise ValueError("Password must be a string.")
17    return credentials.get(user) == password
18 import Task2
19 help(Task2)
20
```

```
PS D:\courses\AIAC\Lab_ass_9.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_9.1/Task2.py
Help on module Task2:

NAME
    Task2

FUNCTIONS
    login(user: str, password: str, credentials: dict) -> bool
        Checks if the provided username and password match the credentials.
        Args:
        -- More --
```

Comparison:

Google style is simple and easy to read, NumPy style is more structured and detailed, and reST style is more formal but harder for beginners.

Recommendation:

Google style is best for new developers because it is clear, beginner-friendly, and easy to understand quickly.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

- o add(a, b) – returns the sum of two numbers

- o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

Code:

```
"""
this module includes four functions and their explanation
and also includes a docstring to explain the usage of each function
"""
def add(a,b):
    """
    This function takes two numbers as input and returns their sum.

    Parameters:
    a (int or float): The first number to be added.
    b (int or float): The second number to be added.

    Returns:
    int or float: The sum of the two input numbers.
    """
    return a + b
def subtract(a,b):
    """
    This function takes two numbers as input and returns their difference.

    Parameters:
    a (int or float): The number from which to subtract.
    b (int or float): The number to be subtracted.

    Returns:
    int or float: The difference of the two input numbers.
    """
    return a - b
def multiply(a,b):
    """
    This function takes two numbers as input and returns their product.

    Parameters:
    a (int or float): The first number to be multiplied.
    b (int or float): The second number to be multiplied.
```

```

Returns:
int or float: The product of the two input numbers.
"""
return a * b
def divide(a,b):
    """
    This function takes two numbers as input and returns their quotient.

    Parameters:
    a (int or float): The dividend.
    b (int or float): The divisor.

    Returns:
    int or float: The quotient of the two input numbers.

    Raises:
    ZeroDivisionError: If the divisor is zero.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b

```

```
python -m pydoc Task3
```

This function takes two numbers as input and returns their sum.

Parameters:

a (int or float): The first number to be added.

b (int or float): The second number to be added.

Returns:

int or float: The sum of the two input numbers.

This function takes two numbers as input and returns their difference.

Parameters:

a (int or float): The number from which to subtract.

b (int or float): The number to be subtracted.

Returns:

int or float: The difference of the two input numbers.

This function takes two numbers as input and returns their product.

Parameters:

a (int or float): The first number to be multiplied.

b (int or float): The second number to be multiplied.

Returns:

int or float: The product of the two input numbers.

This function takes two numbers as input and returns their quotient.

Parameters:

a (int or float): The dividend.

b (int or float): The divisor.

Returns:

int or float: The quotient of the two input numbers.

Raises:

ZeroDivisionError: If the divisor is zero.

Help on module Task3:

NAME

Task3

DESCRIPTION

-- More --

[index](#)

Task3 [d:\courses\aiac\lab_ass_9.1\task3.py](#)

this module includes four functions and their explanation
and also includes a docstring to explain the usage of each function

Functions

add(a, b)

This function takes two numbers as input and returns their sum.

Parameters:

a (int or float): The first number to be added.

b (int or float): The second number to be added.

Returns:

int or float: The sum of the two input numbers.

divide(a, b)

This function takes two numbers as input and returns their quotient.

Parameters:

a (int or float): The dividend.

b (int or float): The divisor.

Returns:

int or float: The quotient of the two input numbers.

Raises:

ZeroDivisionError: If the divisor is zero.

multiply(a, b)

This function takes two numbers as input and returns their product.

Parameters:

a (int or float): The first number to be multiplied.

b (int or float): The second number to be multiplied.

Returns:

int or float: The product of the two input numbers.

subtract(a, b)

This function takes two numbers as input and returns their difference.

Parameters:

a (int or float): The number from which to subtract.

b (int or float): The number to be subtracted.

Returns:

int or float: The difference of the two input numbers.

Problem 4: Conversion Utilities Module

Task:

1. Write a module named `conversion.py` with functions:
 - o `decimal_to_binary(n)`
 - o `binary_to_decimal(b)`
 - o `decimal_to_hexadecimal(n)`
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

Code:

```
"""
this module includes three functions
decimal_to_binary(n), binary_to_decimal(b), decimal_to_hexadecimal(n) and their
explanation
and also includes a docstring to explain the usage of each function
"""
def decimal_to_binary(n):
    """
    This function takes a decimal number as input and returns its binary
    representation as a string.

    Parameters:
    n (int): The decimal number to be converted to binary.

    Returns:
    str: The binary representation of the input decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    return bin(n)[2:]
def binary_to_decimal(b):
    """
    This function takes a binary number as input and returns its decimal
    representation as an integer.

    Parameters:
    b (str): The binary number to be converted to decimal. It should be a
    string of '0's and '1's.

    Returns:
    int: The decimal representation of the input binary number.

    Raises:
    ValueError: If the input is not a valid binary number.
    """
    if not all(char in '01' for char in b):
        raise ValueError("Input must be a valid binary number (string of '0's
and '1's).")
    return int(b, 2)
def decimal_to_hexadecimal(n):
    """
    This function takes a decimal number as input and returns its hexadecimal
    representation as a string.

    Parameters:
    n (int): The decimal number to be converted to hexadecimal.

    Returns:
    str: The hexadecimal representation of the input decimal number.
    """
```



```

    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    return hex(n)[2:].upper()
print(decimal_to_binary.__doc__)
print(binary_to_decimal.__doc__)
print(decimal_to_hexadecimal.__doc__)

import Task4
help(Task4)

```

```
PS D:\courses\AIAC\Lab_ass_9.1> python -m pydoc Task4
```

This function takes a decimal number as input and returns its binary representation as a string.

Parameters:

n (int): The decimal number to be converted to binary.

Returns:

str: The binary representation of the input decimal number.

This function takes a binary number as input and returns its decimal representation as an integer.

Parameters:

b (str): The binary number to be converted to decimal. It should be a string of '0's and '1's.

Returns:

int: The decimal representation of the input binary number.

Raises:

ValueError: If the input is not a valid binary number.

This function takes a decimal number as input and returns its hexadecimal representation as a string.

Parameters:

n (int): The decimal number to be converted to hexadecimal.

Returns:

str: The hexadecimal representation of the input decimal number.

Help on module Task4:

NAME

Task4

DESCRIPTION

-- More --

```
PS D:\courses\AIAC\Lab_ass_9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> |
```

Python 3.14.2 [tags/v3.14.2:df79316, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords

Task4 [index](#)
[d:\courses\aiac\lab_ass_9.1\Task4.py](#)

this module includes three functions `decimal_to_binary(n)`, `binary_to_decimal(b)`, `decimal_to_hexadecimal(n)` and their explanation and also includes a docstring to explain the usage of each function

Modules
[Task4](#)

Functions

binary_to_decimal(b)
This function takes a binary number as input and returns its decimal representation as an integer.
Parameters:
b (str): The binary number to be converted to decimal. It should be a string of '0's and '1's.
Returns:
int: The decimal representation of the input binary number.
Raises:
ValueError: If the input is not a valid binary number.

decimal_to_binary(n)
This function takes a decimal number as input and returns its binary representation as a string.
Parameters:
n (int): The decimal number to be converted to binary.
Returns:
str: The binary representation of the input decimal number.

decimal_to_hexadecimal(n)
This function takes a decimal number as input and returns its hexadecimal representation as a string.
Parameters:
n (int): The decimal number to be converted to hexadecimal.
Returns:
str: The hexadecimal representation of the input decimal number.

Problem 5 – Course Management Module

Task:

1. Create a module `course.py` with functions:
 - o `add_course(course_id, name, credits)`
 - o `remove_course(course_id)`
 - o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

```
Code: """
This module include three functions
add_course(course_id,name,credits),remove_course(course_id),get_course(course_
id) and their explanation
and also includes a docstring to explain the usage of each function
"""
courses = {}
```

```

def add_course(course_id, name, credits):
    """
    This function adds a course to the courses dictionary.

    Parameters:
    course_id (str): The unique identifier for the course.
    name (str): The name of the course.
    credits (int): The number of credits for the course.

    Returns:
    None
    """
    if course_id in courses:
        raise ValueError("Course ID already exists.")
    courses[course_id] = {"name": name, "credits": credits}
def remove_course(course_id):
    """
    This function removes a course from the courses dictionary.
    Parameters:
    course_id (str): The unique identifier for the course to be removed.
    Returns:
    None
    Raises:
    KeyError: If the course ID does not exist in the courses dictionary.
    """
    if course_id not in courses:
        raise KeyError("Course ID does not exist.")
    del courses[course_id]
def get_course(course_id):
    """
    This function retrieves the details of a course from the courses
    dictionary.
    Parameters:
    course_id (str): The unique identifier for the course to be retrieved.
    Returns:
    dict: A dictionary containing the name and credits of the course.
    Raises:
    KeyError: If the course ID does not exist in the courses dictionary.
    """
    if course_id not in courses:
        raise KeyError("Course ID does not exist.")
    return courses[course_id]
print(add_course.__doc__)
print(remove_course.__doc__)
print(get_course.__doc__)
import Task5
help(Task5)

```

```
PS D:\courses\AIAC\lab_ass_9.1> python --pydoc Task5
```

This function adds a course to the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
None

This function removes a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be removed.

Returns:
None

Raises:
KeyError: If the course ID does not exist in the courses dictionary.

This function retrieves the details of a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be retrieved.

Returns:
dict: A dictionary containing the name and credits of the course.

Raises:
KeyError: If the course ID does not exist in the courses dictionary.
Help on module Task5:

NAME
Task5

DESCRIPTION

This module include three functions `add_course(course_id,name,credits)`,`remove_course(course_id)`,`get_course(course_id)` and their explanation and also includes a docstring to explain the usage of each function

FUNCTIONS

`add_course(course_id, name, credits)`

This function adds a course to the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
None

`get_course(course_id)`

This function retrieves the details of a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be retrieved.

Returns:
dict: A dictionary containing the name and credits of the course.

Raises:
KeyError: If the course ID does not exist in the courses dictionary.

`remove_course(course_id)`

This function removes a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be removed.

-- More --

← ↻ ⓘ 127.0.0.1:5500/Task5.html

[index](#)

Task5 [d:\courses\aiac\lab_ass_9.1\Task5.py](#)

This module include three functions `add_course(course_id,name,credits)`,`remove_course(course_id)`,`get_course(course_id)` and their explanation and also includes a docstring to explain the usage of each function

Modules

[Task5](#)

Functions

`add_course(course_id, name, credits)`

This function adds a course to the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
None

`get_course(course_id)`

This function retrieves the details of a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be retrieved.

Returns:
dict: A dictionary containing the name and credits of the course.

Raises:
KeyError: If the course ID does not exist in the courses dictionary.

`remove_course(course_id)`

This function removes a course from the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course to be removed.

Returns:
None
Raises:
KeyError: If the course ID does not exist in the courses dictionary.

Data

`courses = {}`